

Python – Collections, functions and Modules

- Accessing List

1. Understanding how to create and access elements in a list.

a. Lists are ordered, mutable collections, created with square brackets:

Ex :

```
fruits = ["apple", "banana", "cherry", "date"]
#Access individual elements by index:
print(fruits[0])    # "apple"
print(fruits[2])    # "cherry"
```

2. Indexing in lists (positive and negative indexing).

```
fruits = ["apple", "banana", "cherry", "date"]
#           0           1           2           3
#          -4          -3          -2          -1
```

→ Positive Indexing :

◆ starts at 0 from the left

Ex :

```
print(fruits[0])    # "apple"    (first element)
print(fruits[3])    # "date"     (fourth element)|
```

→ Negative Indexing :

◆ starts at -1 from the right (useful for accessing from the end without knowing the length)

Ex :

```
print(fruits[-1])   # "date"     (last element)
print(fruits[-2])   # "cherry"   (second to last)|
```

3. Slicing a list: accessing a range of elements.

- a. `list[start:stop:step]` — start is inclusive, stop is exclusive.

Ex :

```
nums = [10, 20, 30, 40, 50, 60]
```

```
nums[1:4]      # [20, 30, 40]    (indices 1, 2, 3)
nums[:3]       # [10, 20, 30]   (start to index 2)
nums[3:]       # [40, 50, 60]   (index 3 to end)
nums[:]        # [10, 20, 30, 40, 50, 60] (full copy)

nums[-3:]      # [40, 50, 60]   (last 3 elements)
nums[:-2]      # [10, 20, 30, 40] (all except last 2)

nums[::-2]     # [10, 30, 50]   (every 2nd element)
nums[::-1]     # [60, 50, 40, 30, 20, 10] (reversed)
```

• List Operations

1. Common list operations: concatenation, repetition, membership.

→ Concatenation

- ◆ joining lists with `+` (creates a new list)

Ex :

```
list1 = [1, 2, 3]
list2 = [4, 5, 6]
combined = list1 + list2
print(combined)      # [1, 2, 3, 4, 5, 6]
```

→ Repetition

- ◆ repeating a list with `*`

Ex :

```
print([0] * 5)        # [0, 0, 0, 0, 0]
print([1, 2] * 3)     # [1, 2, 1, 2, 1, 2]
```

→ Membership :

- ◆ checking if a value exists with `in` / `not in`

Ex :

```
fruits = ["apple", "banana", "cherry"]

print("banana" in fruits)      # True
print("mango" in fruits)       # False
print("mango" not in fruits)   # True
```

2. Understanding list methods like `append()`, `insert()`, `remove()`, `pop()`.

→ **`append(item)`** :

- ◆ Adds a single item to the end of the list

→ **`insert(index, item)`** :

- ◆ Inserts an item at a specific position, shifting later elements right

→ **`remove(value)`** :

- ◆ Removes the *first* matching value (raises `ValueError` if not found)

→ **`pop(index)`** :

- ◆ Removes and returns the item at the given index (default: last item). Useful when you need the removed value:

Ex :

```
fruits = ["apple", "banana"]

fruits = ["apple", "banana"]
fruits.append("cherry")
print(fruits)    # ["apple", "banana", "cherry"]

fruits.insert(1, "kiwi")
print(fruits)    # ["apple", "kiwi", "banana", "cherry"]

fruits.remove("kiwi")
print(fruits)    # ["apple", "banana", "cherry"]

fruits.remove("mango")    # ValueError: list.remove(x): x not in list
last = fruits.pop()       # removes & returns "cherry"
print(last)               # "cherry"
print(fruits)              # ["apple", "banana"]

first = fruits.pop(0)      # removes & returns "apple"
print(first)               # "apple"
print(fruits)              # ["banana"]
```

● Working with list

1. Iterating over a list using loops.

→ Basic for loop :

- ◆ Iterates over each element directly

Ex

```
fruits = ["apple", "banana", "cherry"]

for fruit in fruits:
    print(fruit)
# apple
# banana
# cherry
```

2. Sorting and reversing a list using sort(), sorted(), and reverse().

→ sort()

- ◆ sorts the list in place (modifies original, returns None):

Ex :

```
nums = [4, 1, 3, 2]
nums.sort()
print(nums)      # [1, 2, 3, 4]

nums.sort(reverse=True)
print(nums)      # [4, 3, 2, 1]
```

→ sorted()

- ◆ returns a new sorted list, original unchanged:

Ex :

```
nums = [4, 1, 3, 2]
result = sorted(nums)
print(result)      # [1, 2, 3, 4]
print(nums)        # [4, 1, 3, 2] - unchanged
```

→ reverse()

- ◆ Reverses the list in place (does not sort, just flips order)

Ex :

```
nums = [1, 2, 3, 4]
nums.reverse()
print(nums)      # [4, 3, 2, 1]
```

3. Basic list manipulations: addition, deletion, updating, and slicing.

```

#Addition
nums = [1, 2, 3]
nums.append(4)           # add single item to end → [1, 2, 3, 4]
nums.insert(0, 0)        # add at specific position → [0, 1, 2, 3, 4]
nums.extend([5, 6])      # add multiple items → [0, 1, 2, 3, 4, 5, 6]

#Deletion
nums = [10, 20, 30, 40, 50]
nums.remove(30)          # remove by value → [10, 20, 40, 50]
nums.pop()               # remove last item → [10, 20, 40]
nums.pop(0)              # remove by index → [20, 40]
nums.clear()             # empty the list → []

#updating
nums = [10, 20, 30]
nums[1] = 99             # update single element → [10, 99, 30]
nums[0:2] = [1, 2]       # update multiple elements via slice → [1, 2, 30]

#slicing
nums = [10, 20, 30, 40, 50]

print(nums[1:4])         # [20, 30, 40]
print(nums[:3])          # [10, 20, 30]
print(nums[-2:])         # [40, 50]
print(nums[::-1])        # [50, 40, 30, 20, 10] - reversed copy

nums[1:3] = [99, 98, 97] # replace a slice with multiple values
print(nums)              # [10, 99, 98, 97, 40, 50]

```

● Tuples

1. Introduction to tuples, immutability.

- a. A tuple is an ordered collection like a list, but immutable — once created, its elements cannot be changed, added, or removed.
- b. Trying to modify a tuple raises an error:
Ex :

```

point = (3, 4)
colors = ("red", "green", "blue")

```

2. Creating and accessing elements in a tuple.

→ Creating Tuple :

```

empty = ()
single = (5,) # comma is required for a one-element tuple!
not_a_tuple = (5) # this is just an int, not a tuple
numbers = (1, 2, 3, 4)
mixed = (1, "two", 3.0)
no_parens = 1, 2, 3 # parentheses are optional

```

→ Accessing elements — same indexing rules as lists:

```
point = (10, 20, 30)

print(point[0])      # 10    (first element)
print(point[-1])     # 30    (last element)|
```

→ Convert between tuple and list:

```
t = (1, 2, 3)
lst = list(t)          # [1, 2, 3]
back_to_tuple = tuple(lst) # (1, 2, 3)|
```

3. Basic operations with tuples: concatenation, repetition, membership.

→ Concatenation — joining tuples with + (creates a new tuple):

Ex :

```
a = (1, 2, 3)
b = (4, 5, 6)
combined = a + b
print(combined)    # (1, 2, 3, 4, 5, 6)|
```

→ Repetition — repeating a tuple with *:

Ex :

```
print((0,) * 5)      # (0, 0, 0, 0, 0)
print((1, 2) * 3)    # (1, 2, 1, 2, 1, 2)
```

→ Membership — checking existence with in / not in:

Ex :

```
colors = ("red", "green", "blue")

print("red" in colors)      # True
print("yellow" in colors)   # False
print("yellow" not in colors) # True
```

- Accessing Tuples

1. Accessing tuple elements using positive and negative indexing.

→ Positive and negative indexing

Ex :

```
point = (10, 20, 30, 40)
#           0     1     2     3
#        -4    -3    -2    -1

print(point[0])      # 10    (first element)
print(point[2])      # 30
print(point[-1])     # 40    (last element)
print(point[-2])     # 30    (second to last)
```

2. Slicing a tuple to access ranges of elements.

- a. `tup[start:stop:step]` — same rules as lists, returns a new tuple.

Ex :

```
nums = (10, 20, 30, 40, 50, 60)

nums[1:4]      # (20, 30, 40)
nums[:3]       # (10, 20, 30)
nums[3:]       # (40, 50, 60)
nums[-3:]      # (40, 50, 60)
nums[::2]       # (10, 30, 50)
nums[::-1]      # (60, 50, 40, 30, 20, 10)
```

- Dictionaries

1. Introduction to dictionaries: key-value pairs.

- a. A dictionary stores data as key-value pairs.

- b. Keys must be unique and immutable (strings, numbers, tuples); values can be anything.

Ex :

```
student = {  
    "name": "Niyati",  
    "age": 26,  
    "major": "Computer Science"  
}
```

2. Accessing, adding, updating, and deleting dictionary elements.

```
student = {  
    "name": "Niyati",  
    "age": 26,  
    "major": "Computer Science"  
}  
  
#accessing  
print(student["name"])    # "Niyati"  
print(student.get("age"))    # 26  
  
#adding/updating  
student["gpa"] = 3.8    # adds new key  
student["major"] = "Msc.IT"    # updates existing key  
  
#deleting  
student.pop("gpa")    # removes and returns the value  
student.clear()    # empties the dictionary
```


3. Dictionary methods like keys(), values(), and items().

```
student = {
    "name": "Niyati",
    "age": 26,
    "major": "CS"
}

print(student.keys())
# dict_keys(['name', 'age', 'major'])

print(student.values())
# dict_values(['Alice', 21, 'CS'])

print(student.items())
# dict_items([('name', 'Niyati'), ('age', 26), ('major', 'CS')])

# convert to lists if needed
print(list(student.keys()))      # ['name', 'age', 'major']
```

- Working with Dictionaries

1. Iterating over a dictionary using loops.

```
student = {
    "name": "Niyati",
    "age": 26,
    "major": "CS"
}

# iterates over keys by default
for key in student:
    print(key)

# iterate over key-value pairs
for key, value in student.items():
    print(f"{key}: {value}")
'''
Output :
name: Niyati
age: 26
major: CS
'''

# iterate over values only
for value in student.values():
    print(value)|
```

2. Merging two lists into a dictionary using loops or zip().

→ Using Loop

```
keys = ["name", "age", "major"]
values = ["niya", 21, "CS"]

result = {}
for k, v in zip(keys, values):
    result[k] = v
print(result)
# {'name': 'niya', 'age': 21, 'major': 'CS'}
```

→ Using Zip

```
keys = ["name", "age", "major"]
values = ["niya", 21, "CS"]

result = dict(zip(keys, values))
print(result)
# {'name': 'niya', 'age': 21, 'major': 'CS'}
```

3. Counting occurrences of characters in a string using dictionaries.

```
text = "banana"
counts = {}

for char in text:
    if char in counts:
        counts[char] += 1
    else:
        counts[char] = 1

print(counts)    # {'b': 1, 'a': 3, 'n': 2}
```

- Functions

1. Defining functions in Python.

- a. def keyword, function name, parentheses (parameters go inside), colon, then an indented body.

```
def greet():
    print("Hello!")

greet()    # Hello!
```

2. Different types of functions: with/without parameters, with/without return values.

→ Without parameters, without return value:

```
def greet():
    print("Hello!")
```

→ With parameters, without return value:

```
def greet(name):
    print(f"Hello, {name}!")

greet("niya")    # Hello, niya!
```

→ With parameters and return value:

```
def add(a, b):
    return a + b

result = add(3, 5)
print(result)    # 8
```

→ Without parameters, with return value:

```
def get_pi():
    return 3.14159
```

→ Default parameter values:

```
def greet(name="Guest"):
    print(f"Hello, {name}!")

greet()    # Hello, Guest!
greet("niya")    # Hello, niya!
```

3. Anonymous functions (lambda functions).

```
x = lambda a,b : a + b
print(x(5,15))
```

```
y = lambda a: "Even" if a % 2 == 0 else "Odd"
print(y(10))|
```

- Modules

1. Introduction to Python modules and importing modules.
 - a. A module is a .py file containing reusable code (functions, classes, variables).

2. Standard library modules: math, random.

→ Random module :

```
import random

random.random()           # random float between 0.0 and 1.0
random.randint(1, 10)     # random integer between 1 and 10 (inclusive)
random.choice([1, 2, 3])  # random element from a list
random.shuffle(my_list)   # shuffles a list in place
random.sample([1,2,3,4], 2) # 2 unique random elements|
```

→ Math Module :

```
import math

math.sqrt(25)             # 5.0
math.pow(2, 3)            # 8.0
math.floor(4.7)           # 4
math.ceil(4.2)            # 5
math.pi                  # 3.14159...
math.factorial(5)         # 120|
```

3. Creating custom modules.

userDefinmodule file :

```

def oddEven(num) :
    if num %2 == 0:
        print(num, "is Even")
    else :
        print(num, "is Odd")

def maxofTwo(num1,num2) :
    if num1>num2 :
        print(num1, "is Max")
    else :
        print(num2, "is Max")|

```

FunctionCallingfile :

```

import userDefineLib

#oddEven
num1 = int(input("Enter Number : "))
userDefineLib.oddEven(num1)
print("-" * 45)
#max 2
num1 = int(input("Enter Number : "))
num2 = int(input("Enter Number : "))
userDefineLib.maxofTwo(num1,num2)|

```